

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Аппаратное и программное обеспечение сетей»

Тема: «Организация ТСП – сервера»

Выполнил:

Студент 3 курса

Группы ИИ-27

Титунин А.А.,

Проверил:

Кулик А.Д.

Вариант 4

Цель: изучить принцип работы TCP и научиться работать с сокетами.

Задание:

8.1) Изучить теоретический материал, функции и классы пакета `java.net` и листинг программы реализации TCP-сервера. Получить индивидуальное задание у преподавателя.

8.2) Разработать программу работы TCP-эхо-сервера, выполняющую функции согласно варианта задания (см. приложения). В качестве клиента использовать программу `telnet`. Выполнить проверку программы согласно методике, приведенной в разделе

8.1) Продемонстрировать работу системы преподавателю. 8.3 Представить отчет, содержащий титульный лист, листинг программы с подробными комментариями основных фрагментов программы.

8.4) Подготовиться к защите лабораторной работы по теоретическому материалу, функциям, собственным результатам, полученным в ходе выполнения лабораторной работы

9.1 Изучить теоретический материал, функции и классы пакета `java.net` и листинг программ реализации TCP-клиента.

9.2 Разработать программу работы TCP-клиента (см. приложения). Выполнить проверку программы с использованием разработанного в предыдущей лабораторной работе TCP-сервера. Продемонстрировать работу системы преподавателю.

9.3 Представить отчет, содержащий титульный лист, листинг программы с подробными комментариями основных фрагментов программы.

9.4 Подготовиться к защите лабораторной работы по теоретическому материалу, функциям, собственным результатам, полученным в ходе выполнения лабораторной работы

Задание варианта:

После приема каждой группы из 64 символов от клиента сервер формирует и отправляет клиенту цепочку <«символ1-количество1», «символ2-количество2» и т.д.>, где «символ» - встреченный в группе символ, а «количество» - их количество в группе. Если в очередной группе число разных символов меньше 3, то отправляется соответствующее сообщение клиенту и сервер разрывает соединение.

Клиент должен отправлять введенные символы по нажатию клавиши `Home`, а также устанавливать соединение при помощи команды `connect` и разрывать его при помощи команды `disconnect`. Клиент должен вести лог отдельным текстовым файлом, куда записываются время установления соединения, время разрыва соединения и информация, переданная от сервера.

Ход работы:

Пользуясь методическими указаниями я написал программу сервер и программу клиент согласно своему варианту на языке `java` в среде разработки `Visual Studio Code` с использованием `Winsock`.

Сервер запускается и открывает соединение, после чего к нему подключается клиент при помощи команды `connect`, и отправляет введенные пользователем символы, которые собираются в блоки по 64 байта на стороне сервера. Далее сервер возвращает количество каждого символа в блоке, или разрывает соединение, если разных символов 3 или меньше.

Листинг программы-сервера:

```
import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.net.*;
import java.util.ArrayList;

class Pair {
    int number;
    char letter;

    Pair(char letter, int number) {
        this.letter = letter;
        this.number = number;
    }
}

public class Server {
    public static final int PORT = 8080;
    public static final int BUFFER_SIZE = 64;

    private JFrame frame;
    private JTextArea log;
    private JLabel status;
    private JButton startButton;
    private JButton stopButton;

    private ServerSocket serverSocket;
    private volatile boolean running = false;
    private Thread serverThread;

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Server().buildUI());
    }

    private void buildUI() {
        frame = new JFrame("TCP Server");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        log = new JTextArea(22, 55);
        log.setEditable(false);
        log.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));

        status = new JLabel("Status: stopped");
        startButton = new JButton("Start");
        stopButton = new JButton("Stop");
        stopButton.setEnabled(false);

        startButton.addActionListener(e -> startServer());
        stopButton.addActionListener(e -> stopServer());

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.LEFT));
        buttons.add(startButton);
        buttons.add(stopButton);
        buttons.add(status);

        frame.getContentPane().setLayout(new BorderLayout(5, 5));
        frame.getContentPane().add(new JScrollPane(log), BorderLayout.CENTER);
        frame.getContentPane().add(buttons, BorderLayout.SOUTH);

        frame.pack();
        frame.setLocationRelativeTo(null);
    }
}
```

```

frame.setVisible(true);

appendLog("[SERVER] Ready. Press Start to listen on port " + PORT + ".");
}

private void startServer() {
    if (running) return;

    try {
        serverSocket = new ServerSocket(PORT);
    } catch (IOException e) {
        appendLog("[SERVER] Cannot open port " + PORT + ": " + e.getMessage());
        return;
    }

    running = true;
    startButton.setEnabled(false);
    stopButton.setEnabled(true);
    status.setText("Status: listening on port " + PORT);
    appendLog("[SERVER] Started: " + serverSocket);

    serverThread = new Thread(this::acceptLoop);
    serverThread.setDaemon(true);
    serverThread.start();
}

private void acceptLoop() {
    while (running) {
        try {
            Socket socket = serverSocket.accept();
            appendLog("[SERVER] Connection accepted: " + socket);

            // Каждого клиента обрабатываем в отдельном потоке,
            // чтобы сервер мог принимать нескольких сразу
            Thread clientThread = new Thread(() -> handleClient(socket));
            clientThread.setDaemon(true);
            clientThread.start();

        } catch (IOException e) {
            if (running) {
                appendLog("[SERVER] Accept error: " + e.getMessage());
            }
            // если сокет закрыт — выходим из цикла
            if (serverSocket.isClosed()) break;
        }
    }
}

private void handleClient(Socket socket) {
    try {
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(
            new BufferedWriter(
                new OutputStreamWriter(socket.getOutputStream()), true));

        char[] buffer = new char[BUFFER_SIZE];
        int totalRead = 0;
        ArrayList<Pair> p = new ArrayList<>();

        int ch;
        while ((ch = in.read()) != -1) {

```

```

        buffer[totalRead++] = (char) ch;

        if (totalRead == BUFFER_SIZE) {
            // Считаем вхождения
            for (char c : buffer) {
                boolean isExist = false;
                for (int i = 0; i < p.size(); i++) {
                    if (p.get(i).letter == c) {
                        p.get(i).number++;
                        isExist = true;
                        break;
                    }
                }
                if (!isExist) {
                    p.add(new Pair(c, 1));
                }
            }

            appendLog("[SERVER] Block from " + socket.getPort()
                + ": " + new String(buffer));

            if (p.size() <= 3) {
                appendLog("[SERVER] -> not enough characters, closing client "
                    + socket.getPort());
                out.println("not enough characters");
                out.println("closing...");
                socket.close();
                return;
            } else {
                appendLog("[SERVER] -> answer:");
                for (int i = 0; i < p.size(); i++) {
                    String line = "Character " + p.get(i).letter
                        + ": " + p.get(i).number;
                    appendLog("[SERVER]   " + line);
                    out.println(line);
                }
                out.flush();
            }

            totalRead = 0;
            p.clear();
        }
    }

    appendLog("[SERVER] Client " + socket.getPort() + " disconnected.");

    } catch (IOException e) {
        appendLog("[SERVER] Client error: " + e.getMessage());
    } finally {
        try { socket.close(); } catch (IOException ignored) {}
    }
}

private void stopServer() {
    if (!running) return;
    running = false;

    try {
        if (serverSocket != null && !serverSocket.isClosed()) {
            serverSocket.close();
        }
    } catch (IOException ignored) {}
}

```

```

        startButton.setEnabled(true);
        stopButton.setEnabled(false);
        status.setText("Status: stopped");
        appendLog("[SERVER] Stopped.");
    }

    private void appendLog(String line) {
        SwingUtilities.invokeLater(() -> {
            log.append(line + "\n");
            log.setCaretPosition(log.getDocument().getLength());
        });
    }
}

```

Листинг программы-клиента:

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Client {
    private Socket socket;
    private BufferedReader in;
    private PrintWriter out;

    private JTextField inputField;
    private JTextArea log;
    private JLabel status;

    private static final String LOG_FILE = "client_log.txt";
    private static final DateTimeFormatter TIME_FMT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
    private PrintWriter fileLog;

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Client().buildUI());
    }

    private void buildUI() {
        try {
            fileLog = new PrintWriter(new BufferedWriter(
                new FileWriter(LOG_FILE, true)), true);
        } catch (IOException e) {
            System.err.println("Cannot open log file: " + e.getMessage());
        }

        JFrame frame = new JFrame("TCP Client");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        inputField = new JTextField(40);
        log = new JTextArea(20, 55);
        log.setEditable(false);
        log.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));
        status = new JLabel("Status: disconnected");

        JLabel hint = new JLabel(
            "Commands: connect <host> <port> | disconnect | exit  "
            + "— other input is sent to server. Press HOME to execute/send.");
    }
}

```

```

inputField.addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_HOME) {
            handleInput();
            e.consume();
        }
    }
});

inputField.addActionListener(e -> handleInput());

JPanel top = new JPanel(new BorderLayout(5, 5));
top.add(inputField, BorderLayout.NORTH);
top.add(hint, BorderLayout.SOUTH);

frame.getContentPane().setLayout(new BorderLayout(5, 5));
frame.getContentPane().add(top, BorderLayout.NORTH);
frame.getContentPane().add(new JScrollPane(log), BorderLayout.CENTER);
frame.getContentPane().add(status, BorderLayout.SOUTH);

frame.addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        closeEverything();
    }
});

frame.pack();
frame.setLocationRelativeTo(null);
frame.setVisible(true);

appendLog("[CLIENT] Ready. Use 'connect <host> <port>' to connect.");
inputField.requestFocusInWindow();
}

// ----- Разбор ввода -----

private void handleInput() {
    String line = inputField.getText().trim();
    inputField.setText("");
    if (line.isEmpty()) return;

    String[] parts = line.split("\\s+");
    String first = parts[0].toLowerCase();

    if (first.equals("connect")) {
        handleConnect(parts);
    } else if (first.equals("disconnect")) {
        handleDisconnect();
    } else if (first.equals("exit")) {
        handleDisconnect();
        appendLog("[CLIENT] Bye.");
        closeEverything();
        System.exit(0);
    } else {
        sendData(line);
    }

    inputField.requestFocusInWindow();
}

```

```
// ----- Команды -----

private void handleConnect(String[] parts) {
    if (socket != null && !socket.isClosed()) {
        appendLog("[CLIENT] Already connected to " + socket.getRemoteSocketAddress()
            + ". Use 'disconnect' first.");
        return;
    }

    if (parts.length != 3) {
        appendLog("[CLIENT] Usage: connect <host> <port>");
        return;
    }

    String host = parts[1];
    int port;
    try {
        port = Integer.parseInt(parts[2]);
    } catch (NumberFormatException e) {
        appendLog("[CLIENT] Invalid port: " + parts[2]);
        return;
    }

    try {
        socket = new Socket(host, port);
        in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        out = new PrintWriter(
            new BufferedWriter(new OutputStreamWriter(socket.getOutputStream())), true);

        appendLog("[CLIENT] Connected to " + socket);
        status.setText("Status: connected to " + socket.getRemoteSocketAddress());

        // ФАЙЛ: только время соединения
        writeFileLog("CONNECTION START: " + LocalDateTime.now().format(TIME_FMT));

        startReaderThread();

    } catch (IOException e) {
        appendLog("[CLIENT] Cannot connect: " + e.getMessage());
        socket = null;
        in = null;
        out = null;
    }
}

private void handleDisconnect() {
    if (socket == null || socket.isClosed()) {
        appendLog("[CLIENT] Not connected.");
        return;
    }

    try {
        // ФАЙЛ: только время разъединения
        writeFileLog("CONNECTION END: " + LocalDateTime.now().format(TIME_FMT));

        socket.close();
        appendLog("[CLIENT] Disconnected.");
    } catch (IOException e) {
        appendLog("[CLIENT] Error on disconnect: " + e.getMessage());
    } finally {
        socket = null;
        in = null;
    }
}
```

```

        out = null;
        status.setText("Status: disconnected");
    }
}

// ----- Данные -----

private void sendData(String text) {
    if (socket == null || socket.isClosed()) {
        appendLog("[CLIENT] Not connected. Use 'connect <host> <port>' first.");
        return;
    }
    out.print(text);
    out.flush();
    appendLog("[CLIENT] Sent " + text.length() + " characters: " + text);
}

// ----- Чтение ответов -----

private void startReaderThread() {
    final Socket currentSocket = socket;
    final BufferedReader currentIn = in;

    Thread t = new Thread(() -> {
        try {
            String response;
            while ((response = currentIn.readLine()) != null) {
                final String r = response;
                SwingUtilities.invokeLater(() -> appendLog("[SERVER] " + r));

                // ФАЙЛ: только то, что вернул сервер
                writeFileLog("SERVER: " + r);

                if (response.equals("closing...")) {
                    SwingUtilities.invokeLater(() -> {
                        appendLog("[CLIENT] Server closed the connection.");
                        status.setText("Status: disconnected");
                    });
                    // ФАЙЛ: разъединение по инициативе сервера
                    writeFileLog("CONNECTION END: "
                        + LocalDateTime.now().format(TIME_FMT));
                    try { currentSocket.close(); } catch (IOException ignored) {}
                    socket = null;
                    in = null;
                    out = null;
                    return;
                }
            }
        } catch (IOException e) {
            if (currentSocket.isClosed() || socket == null) {
                // закрытие по нашей инициативе — в файл не пишем
            } else {
                SwingUtilities.invokeLater(() ->
                    appendLog("[CLIENT] Read error: " + e.getMessage()));
            }
        }
    });
    t.setDaemon(true);
    t.start();
}

// ----- Логи -----

```

```

private void appendLog(String line) {
    SwingUtilities.invokeLater(() -> {
        log.append(line + "\n");
        log.setCaretPosition(log.getDocument().getLength());
    });
}

private synchronized void writeFileLog(String line) {
    if (fileLog == null) return;
    fileLog.println(line);
}

private void closeEverything() {
    if (socket != null && !socket.isClosed()) {
        writeFileLog("CONNECTION END: " + LocalDateTime.now().format(TIME_FMT));
        try { socket.close(); } catch (IOException ignored) {}
    }
    if (fileLog != null) {
        fileLog.close();
        fileLog = null;
    }
}
}

```

Результат работы программ:

The screenshot displays three windows from a Java application:

- TCP Client:** Shows a list of commands (connect, disconnect, exit) and a log of client actions. The status at the bottom is "disconnected".
- TCP Server:** Shows server status and log output. The status at the bottom is "listening on port 8080".
- client_log - Блокнот:** A Notepad window showing the contents of the log file, including connection start/end times and character counts for each connection.

TCP Client Log:

```

[CLIENT] Ready. Use 'connect <host> <port>' to connect.
[CLIENT] Connected to Socket[addr=localhost/127.0.0.1,port=8080,localport=60179]
[CLIENT] Sent 20 characters: qqqqqqqqqqqqqqqqqqqqq
[CLIENT] Sent 30 characters: wwwwwwwwwwwwwwwwwwwwwwwwwww
[CLIENT] Sent 10 characters: eeeeeeeeee
[CLIENT] Sent 4 characters: rrrr
[SERVER] Character q: 20
[SERVER] Character w: 30
[SERVER] Character e: 10
[SERVER] Character r: 4
[CLIENT] Disconnected.

```

TCP Server Log:

```

[SERVER] Ready. Press Start to listen on port 8080.
[SERVER] Started: ServerSocket[addr=0.0.0.0/0.0.0.0,1
[SERVER] Connection accepted: Socket[addr=/127.0.0.1,
[SERVER] Block from 60179: qqqqqqqqqqqqqqqqqqqwww
[SERVER] -> answer:
[SERVER] Character q: 20
[SERVER] Character w: 30
[SERVER] Character e: 10
[SERVER] Character r: 4
[SERVER] Client 60179 disconnected.

```

client_log - Блокнот:

```

CONNECTION START: 2026-09-29 13:06:28
SERVER: Character e: 13
SERVER: Character r: 14
SERVER: Character f: 15
SERVER: Character d: 15
SERVER: Character q: 7
CONNECTION END: 2026-09-29 13:06:56
CONNECTION START: 2026-09-29 13:35:15
SERVER: Character q: 20
SERVER: Character w: 30
SERVER: Character e: 10
SERVER: Character r: 4
CONNECTION END: 2026-09-29 13:36:01

```

Вывод: в ходе выполнения лабораторной работы я познакомился с ТСП и научился работать с сокетами.